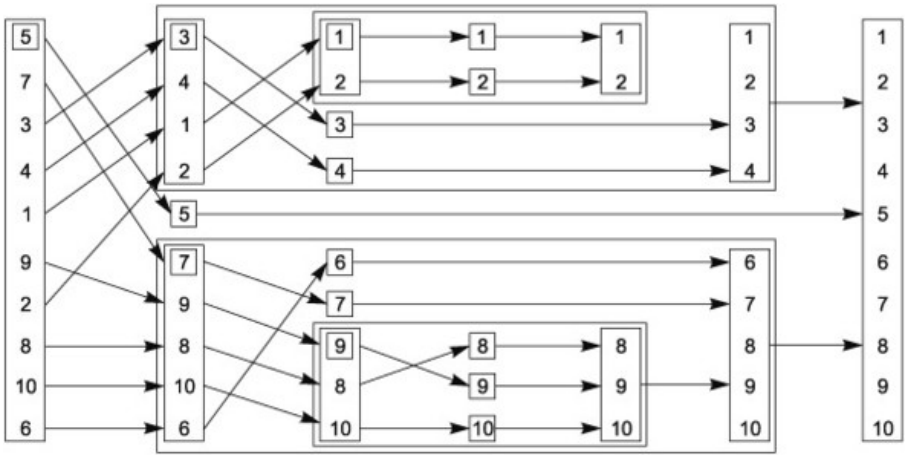
The benefits of functional programming aren't limited to those languages where it's the default paradigm, however. C++ is a multiparadigm language, and it's entirely possible to write programs in the FP style. This is even easier in C++11 than it was in C++98, with the advent of lambda functions (see appendix A, section A.6), the incorporation of `std::bind` from Boost and TR1, and the introduction of automatic type deduction for variables (see appendix A, section A.7). Futures are the final piece of the puzzle that makes FP-style concurrency viable in C++; a future can be passed around between threads to allow the result of one computation to depend on the result of another, *without any explicit access to shared data*.

FP-Style Quicksort

To illustrate the use of futures for FP-style concurrency, let's look at a simple implementation of the Quicksort algorithm. The basic idea of the algorithm is simple: given a list of values, take an element to be the pivot element, and then partition the list into two sets—those less than the pivot and those greater than or equal to the pivot. A sorted copy of the list is obtained by sorting the two sets and returning the sorted list of values less than the pivot, followed by the pivot, followed by the sorted list of values greater than or equal to the pivot. Figure 4.2 shows how a list of 10 integers is sorted under this scheme. An FP-style sequential implementation is shown in the following listing; it takes and returns a list by value rather than sorting in place like `std::sort()` does.

Figure 4.2. FP-style recursive sorting



Listing 4.12. A sequential implementation of Quicksort